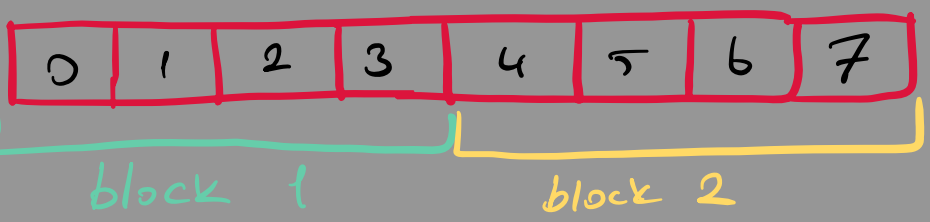
-> big array of data

-> chop into equal size blocks

-> one little worker per block

-> all workers run at the same time in GPU



CUDA

block 0 -> 4 threads (scalars)

thread 0: z[0] = x[0] + y[0]

⋮

thread 4: z[4] = x[4] + y[4]

TRITON

block 0 -> 1 program (vectors)

program (pid = 0)

z[0:4] = x[0:4] + y[0:4]

one vectorized op on the block

the normal way: with loop copy one by one

the GPU way: 3 workers work at the same time for 3 things

↳ give same instruction (and) each worker -> kernel

give badge 0, 1, 2 -> pid

which boxes? = pid x bs + [0,1] -> offs

and what to put

grid -> how many workers to hire

this is the whole idea

"All at the same moment"

entire point of "mask" existing is because of "cdiv"

T T T T | T T F F

0, 1, 2, 3, | 4, 5, -, -

mask = offs < n

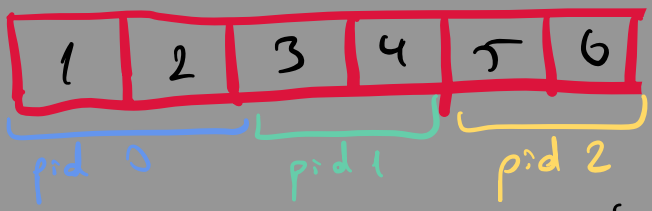
-> is this box full OR belongs to array?

tl.load -> read

tl.store → write to memory.

pid → offs → mask → load → store

EXAMPLE 1: Copy



offs = pid * bs + tl.arrange(0, bs)

↳ 2x2 + [0,1] = [4,5]

0x2 + [0,1] = [0,1]

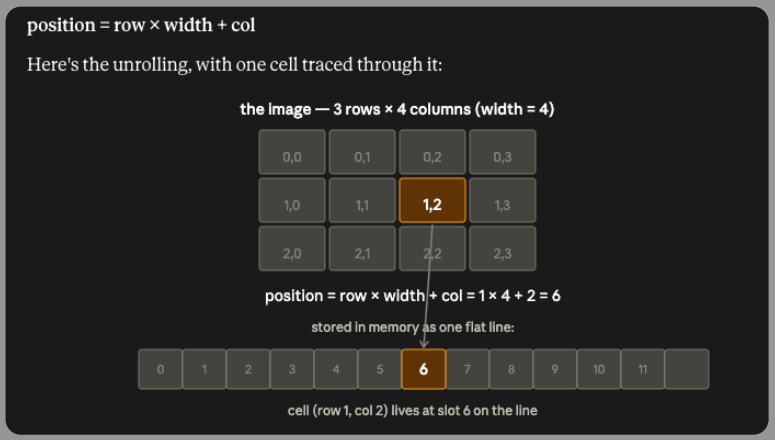
1x2 + [0,1] = [2,3]

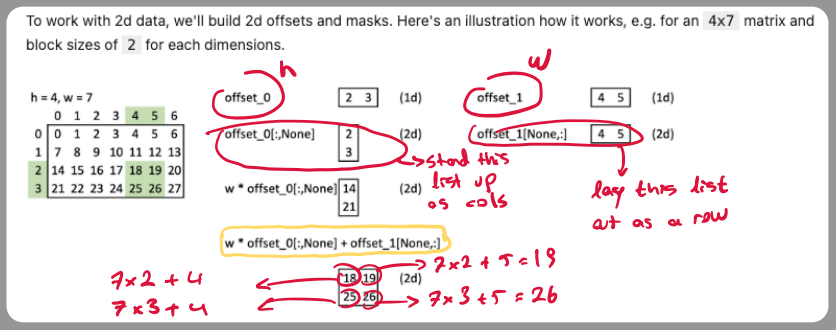
=> so 1, 2, 3, 4, 5, 6

EXAMPLE 2: Grayscaling Image

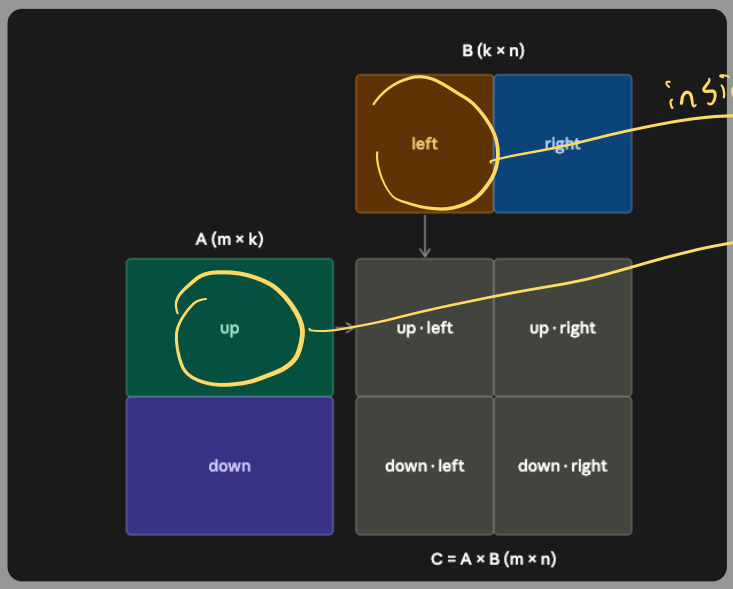
New idea #1: memory is a flat line even for 2D img

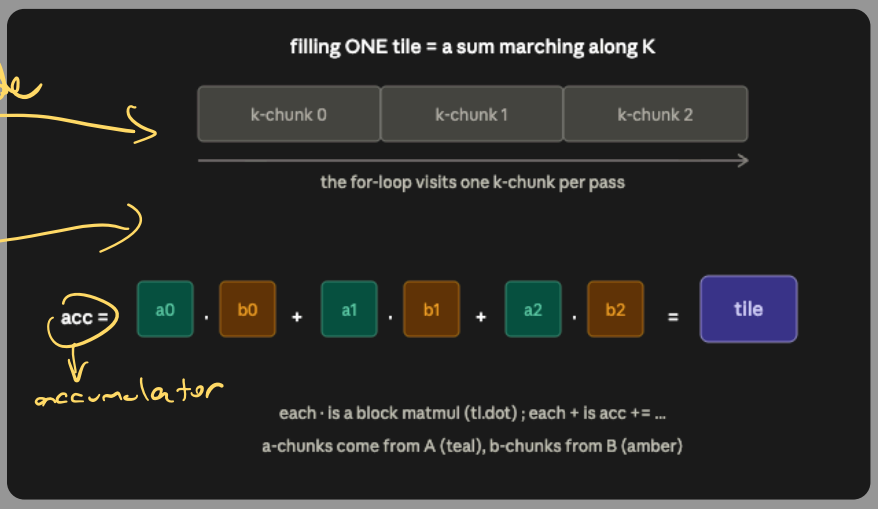
New idea #2: with 2D data, every worker have 2 badges





EXAMPLE 3: Mat Mul (naive)





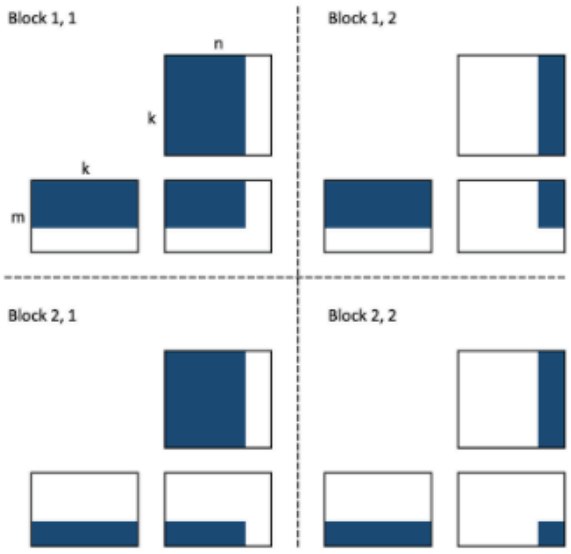
pid → offs → (load → dot → accumulate, repeated) → store

We want to multiply the `m x k`-matrix `A` and the `k x n`-matrix `B` into the `m x n`-matrix `C`.

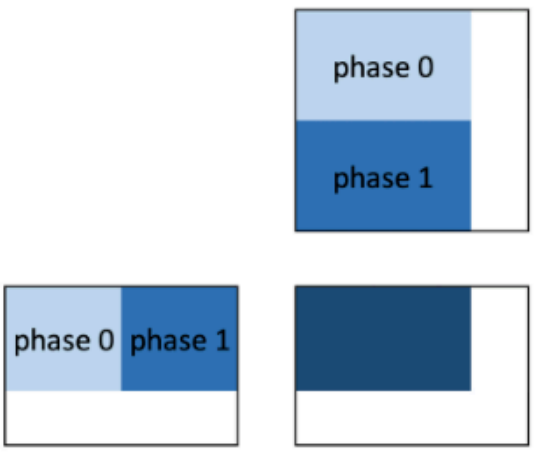
We split the computation along each of the three axes:

- along the m axis - we'll use block dimension 0 to represent this pid_m = program_id(0)
- along the n axis - we'll use block dimension 1 to represent this pid_n = program_id(1)
- along the shared k axis - this will not be represented by a block. All chunks of computation will be done in same block.

Splits along m- and n-dimensions are represented by blocks



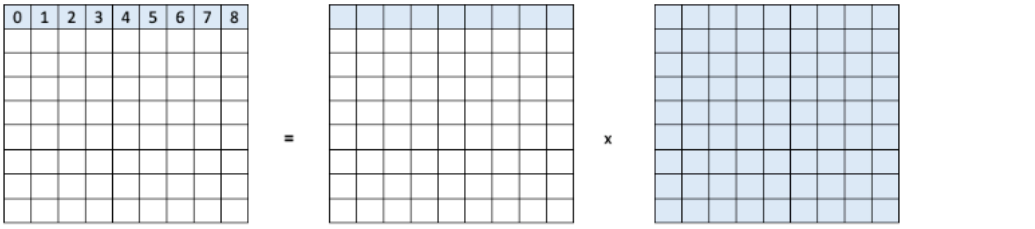
Split along k-dimension is represented by 'phases', which are done in the same block



EXAMPLE 4: fast MatMul

With **naive ordering**

To compute 9 blocks of C we need to load **9 blocks** from A and **81 blocks** from B, so **90 in total**.



With **grouped ordering**

To compute 9 blocks of C we need to load **27 blocks** from A and **27 blocks** from B, so **54 in total**.